

This works by defining an anonymous enumeration containing a single enumerator. The name of the enumerator is made unique (within the translation unit) by “gluing” a fixed prefix such as `g_assert_fail_` to a unique suffix—in this case, the line number on which the `STATIC_ASSERT()` macro is invoked. The value of the enumerator is set to `1 / (!! (expr))`. The double negation `!!` ensures that `expr` has a Boolean value. This value is then cast to an `int`, yielding either 1 or 0 depending on whether the expression is `true` or `false`, respectively. If the expression is `true`, the enumerator will be set to the value `1/1` which is one. But if the expression is `false`, we’ll be asking the compiler to set the enumerator to the value `1/0` which is illegal, and will trigger a compile error.

When our `STATIC_ASSERT()` macro as defined above fails, Visual Studio 2015 produces a compile-time error message like this:

```
1>test.cpp(48): error C2131: expression did not evaluate to
                    a constant
1> test.cpp(48): note: failure was caused by an undefined
                    arithmetic operation
```

Here’s another way to define `STATIC_ASSERT()` using template specialization. In this example, we first check to see if we’re using C++11 or beyond. If so, we use the standard library’s implementation of `static_assert()` for maximum portability. Otherwise we fall back to our custom implementation.

```
#ifndef __cplusplus
    #if __cplusplus >= 201103L
        #define STATIC_ASSERT(expr) \
            static_assert(expr, \
                "static assert failed:" \
                #expr)
    #else
        // declare a template but only define the
        // true case (via specialization)
        template<bool> class TStaticAssert;
        template<> class TStaticAssert<true> {};

        #define STATIC_ASSERT(expr) \
            enum \
            { \
                ASSERT_GLUE(g_assert_fail_, __LINE__) \
                = sizeof(TStaticAssert<!!(expr)>) \
            }
    #endif
#endif
```

This implementation, using template specialization, may be preferable to the previous one using division by zero, because it produces a slightly better error message in Visual Studio 2015:

```
1>test.cpp(48): error C2027: use of undefined type
               'TStaticAssert<false>'
1>test.cpp(48): note: see declaration of
               'TStaticAssert<false>'
```

However, each compiler handles error reporting differently, so your mileage may vary. For more implementation ideas for compile-time assertions, one good reference is http://www.pixelbeat.org/programming/gcc/static_assert.html.

3.3 Data, Code and Memory Layout

3.3.1 Numeric Representations

Numbers are at the heart of everything that we do in game engine development (and software development in general). Every software engineer should understand how numbers are represented and stored by a computer. This section will provide you with the basics you'll need throughout the rest of the book.

3.3.1.1 Numeric Bases

People think most naturally in *base ten*, also known as *decimal notation*. In this notation, ten distinct digits are used (0 through 9), and each digit from right to left represents the next highest power of 10. For example, the number 7803 = $(7 \times 10^3) + (8 \times 10^2) + (0 \times 10^1) + (3 \times 10^0) = 7000 + 800 + 0 + 3$.

In computer science, mathematical quantities such as integers and real-valued numbers need to be stored in the computer's memory. And as we know, computers store numbers in *binary* format, meaning that only the two digits 0 and 1 are available. We call this a *base-two* representation, because each digit from right to left represents the next highest power of 2. Computer scientists sometimes use a prefix of "0b" to represent binary numbers. For example, the binary number 0b1101 is equivalent to decimal 13, because $0b1101 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 8 + 4 + 0 + 1 = 13$.

Another common notation popular in computing circles is *hexadecimal*, or *base 16*. In this notation, the 10 digits 0 through 9 and the six letters A through F are used; the letters A through F replace the decimal values 10 through 15, respectively. A prefix of "0x" is used to denote hex numbers in the C and C++

programming languages. This notation is popular because computers generally store data in groups of 8 bits known as *bytes*, and since a single hexadecimal digit represents 4 bits exactly, a *pair* of hex digits represents a byte. For example, the value $0xFF = 0b11111111 = 255$ is the largest number that can be stored in 8 bits (1 byte). Each digit in a hexadecimal number, from right to left, represents the next power of 16. So, for example, $0xB052 = (11 \times 16^3) + (0 \times 16^2) + (5 \times 16^1) + (2 \times 16^0) = (11 \times 4096) + (0 \times 256) + (5 \times 16) + (2 \times 1) = 45,138$.

3.3.1.2 Signed and Unsigned Integers

In computer science, we use both signed and unsigned integers. Of course, the term “unsigned integer” is actually a bit of a misnomer—in mathematics, the *whole numbers* or *natural numbers* range from 0 (or 1) up to positive infinity, while the *integers* range from negative infinity to positive infinity. Nevertheless, we’ll use computer science lingo in this book and stick with the terms “signed integer” and “unsigned integer.”

Most modern personal computers and game consoles work most easily with integers that are 32 bits or 64 bits wide (although 8- and 16-bit integers are also used a great deal in game programming as well). To represent a 32-bit unsigned integer, we simply encode the value using binary notation (see above). The range of possible values for a 32-bit unsigned integer is $0x00000000$ (0) to $0xFFFFFFFF$ (4,294,967,295).

To represent a *signed* integer in 32 bits, we need a way to differentiate between positive and negative values. One simple approach called the *sign and magnitude* encoding reserves the most significant bit as a *sign bit*. When this bit is zero, the value is positive, and when it is one, the value is negative. This leaves us 31 bits to represent the magnitude of the value, effectively cutting the range of possible magnitudes in half (but allowing both positive and negative forms of every distinct magnitude, including zero).

Most microprocessors use a slightly more efficient technique for encoding negative integers, called *two’s complement* notation. This notation has only one representation for the value zero, as opposed to the two representations possible with simple sign bit (positive zero and negative zero). In 32-bit two’s complement notation, the value $0xFFFFFFFF$ is interpreted to mean -1 , and negative values count down from there. Any value with the most significant bit set is considered negative. So values from $0x00000000$ (0) to $0x7FFFFFFF$ (2,147,483,647) represent positive integers, and $0x80000000$ ($-2,147,483,648$) to $0xFFFFFFFF$ (-1) represent negative integers.

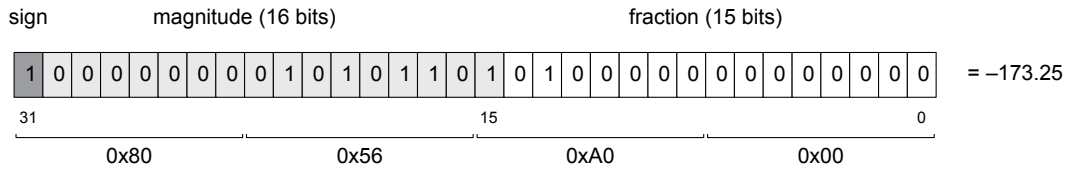


Figure 3.4. Fixed-point notation with 16-bit magnitude and 16-bit fraction.

3.3.1.3 Fixed-Point Notation

Integers are great for representing whole numbers, but to represent fractions and irrational numbers we need a different format that expresses the concept of a decimal point.

One early approach taken by computer scientists was to use *fixed-point* notation. In this notation, one arbitrarily chooses how many bits will be used to represent the whole part of the number, and the rest of the bits are used to represent the fractional part. As we move from left to right (i.e., from the most significant bit to the least significant bit), the magnitude bits represent decreasing powers of two (... , 16, 8, 4, 2, 1), while the fractional bits represent decreasing *inverse* powers of two ($\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{16}$, ...). For example, to store the number -173.25 in 32-bit fixed-point notation with one sign bit, 16 bits for the magnitude and 15 bits for the fraction, we first convert the sign, the whole part and the fractional part into their binary equivalents individually (negative = 0b1, 173 = 0b0000000010101101 and $0.25 = \frac{1}{4} = 0b010000000000000$). Then we pack those values together into a 32-bit integer. The final result is 0x8056A000. This is illustrated in Figure 3.4.

The problem with fixed-point notation is that it constrains both the range of magnitudes that can be represented and the amount of precision we can achieve in the fractional part. Consider a 32-bit fixed-point value with 16 bits for the magnitude, 15 bits for the fraction and a sign bit. This format can only represent magnitudes up to $\pm 65,535$, which isn't particularly large. To overcome this problem, we employ a *floating-point* representation.

3.3.1.4 Floating-Point Notation

In floating-point notation, the position of the decimal place is arbitrary and is specified with the help of an exponent. A floating-point number is broken into three parts: the *mantissa*, which contains the relevant digits of the number on both sides of the decimal point, the *exponent*, which indicates where in that string of digits the decimal point lies, and a *sign bit*, which of course indicates whether the value is positive or negative. There are all sorts of different ways

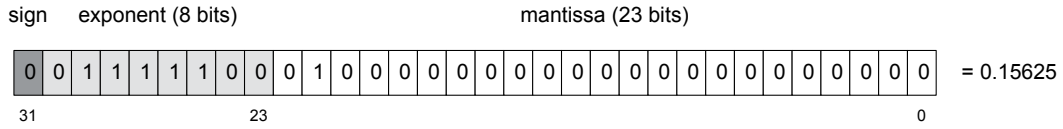


Figure 3.5. IEEE-754 32-bit floating-point format.

to lay out these three components in memory, but the most common standard is IEEE-754. It states that a 32-bit floating-point number will be represented with the sign in the most significant bit, followed by 8 bits of exponent and finally 23 bits of mantissa.

The value v represented by a sign bit s , an exponent e and a mantissa m is $v = s \times 2^{(e-127)} \times (1 + m)$.

The sign bit s has the value $+1$ or -1 . The exponent e is biased by 127 so that negative exponents can be easily represented. The mantissa begins with an implicit 1 that is not actually stored in memory, and the rest of the bits are interpreted as inverse powers of two. Hence the value represented is really $1 + m$, where m is the fractional value stored in the mantissa.

For example, the bit pattern shown in Figure 3.5 represents the value 0.15625, because $s = 0$ (indicating a positive number), $e = 0b01111100 = 124$ and $m = 0b0100\dots = 0 \times 2^{-1} + 1 \times 2^{-2} = \frac{1}{4}$. Therefore,

$$\begin{aligned}
 v &= s \times 2^{(e-127)} \times (1 + m) \\
 &= (+1) \times 2^{(124-127)} \times (1 + \frac{1}{4}) \\
 &= 2^{-3} \times \frac{5}{4} \\
 &= \frac{1}{8} \times \frac{5}{4} \\
 &= 0.125 \times 1.25 = 0.15625.
 \end{aligned}$$

The Trade-Off between Magnitude and Precision

The *precision* of a floating-point number increases as the *magnitude* decreases, and vice versa. This is because there are a fixed number of bits in the mantissa, and these bits must be shared between the whole part and the fractional part of the number. If a large percentage of the bits are spent representing a large magnitude, then a small percentage of bits are available to provide fractional precision. In physics the term *significant digits* is typically used to describe this concept (http://en.wikipedia.org/wiki/Significant_digits).

To understand the trade-off between magnitude and precision, let's look at the largest possible floating-point value, $\text{FLT_MAX} \approx 3.403 \times 10^{38}$, whose

representation in 32-bit IEEE floating-point format is 0x7F7FFFFF. Let's break this down:

- The largest absolute value that we can represent with a 23-bit mantissa is 0x00FFFFFF in hexadecimal, or 24 consecutive binary ones—that's 23 ones in the mantissa, plus the implicit leading one.
- An exponent of 255 has a special meaning in the IEEE-754 format—it is used for values like not-a-number (NaN) and infinity—so it cannot be used for regular numbers. Hence the maximum eight-bit exponent is actually 254, which translates into 127 after subtracting the implicit bias of 127.

So `FLT_MAX` is $0x00FFFFFF \times 2^{127} = 0xFFFFFFFF000000000000000000000000$. In other words, our 24 binary ones were shifted up by 127 bit positions, leaving $127 - 23 = 104$ binary zeros (or $104/4 = 26$ hexadecimal zeros) after the least significant digit of the mantissa. Those trailing zeros don't correspond to any actual bits in our 32-bit floating-point value—they just appear out of thin air because of the exponent. If we were to subtract a small number (where "small" means any number composed of fewer than 26 hexadecimal digits) from `FLT_MAX`, the result would still be `FLT_MAX`, because those 26 least significant hexadecimal digits don't really exist!

The opposite effect occurs for floating-point values whose magnitudes are much less than one. In this case, the exponent is large but negative, and the significant digits are shifted in the opposite direction. We trade the ability to represent large magnitudes for high precision. In summary, we always have the same number of *significant digits* (or really *significant bits*) in our floating-point numbers, and the exponent can be used to shift those significant bits into higher or lower ranges of magnitude.

Subnormal Values

Another subtlety to notice is that there is a finite gap between zero and the smallest nonzero value we can represent with floating-point notation (as it has been described thus far). The smallest nonzero magnitude we can represent is $\text{FLT_MIN} = 2^{-126} \approx 1.175 \times 10^{-38}$, which has a binary representation of 0x00800000 (i.e., the exponent is 0x01, or -126 after subtracting the bias, and the mantissa is all zeros except for the implicit leading one). The next smallest valid value is zero, so there is a finite gap between the values $-\text{FLT_MIN}$ and $+\text{FLT_MIN}$. This underscores the fact that the real number line is *quantized* when using a floating-point representation. (Note that the C++ standard library exposes `FLT_MIN` as the rather more verbose `std::numeric_limits`

`<float>::min()`. We'll stick to `FLT_MIN` in this book for brevity.)

The gap around zero can be filled by employing an extension to the floating-point representation known as *denormalized values*, also known as *subnormal values*. With this extension, any floating-point value with a biased exponent of 0 is interpreted as a subnormal number. The exponent is treated as if it had been a 1 instead of a 0, and the implicit leading 1 that normally sits in front of the bits of the mantissa is changed to a 0. This has the effect of filling the gap between `-FLT_MIN` and `+FLT_MIN` with a linear sequence of evenly-spaced subnormal values. The positive subnormal `float` that is closest to zero is represented by the constant `FLT_TRUE_MIN`.

The benefit of using subnormal values is that it provides greater precision near zero. For example, it ensures that the following two expressions are equivalent, even for values of `a` and `b` that are very close to `FLT_MIN`:

```
if (a == b) { ... }
if (a - b == 0.0f) { ... }
```

Without subnormal values, the expression `a - b` could evaluate to zero even when `a != b`.

Machine Epsilon

For a particular floating-point representation, the *machine epsilon* is defined to be the smallest floating-point value ϵ that satisfies the equation, $1 + \epsilon \neq 1$. For an IEEE-754 floating-point number, with its 23 bits of precision, the value of ϵ is 2^{-23} , which is approximately 1.192×10^{-7} . The most significant digit of ϵ falls *just inside* the range of significant digits in the value 1.0, so adding any value smaller than ϵ to 1.0 has no effect. In other words, any new bits contributed adding a value smaller than ϵ will get “chopped off” when we try to fit the sum into a mantissa with only 23 bits.

Units in the Last Place (ULP)

Consider two floating-point numbers which are identical in all respects except for the value of the least-significant bit in their mantissas. These two values are said to differ by one *unit in the last place* (1 ULP). The actual value of 1 ULP changes depending on the exponent. For example, the floating-point value `1.0f` has an unbiased exponent of zero, and a mantissa in which all bits are zero (except for the implicit leading 1). At this exponent, 1 ULP is equal to the machine epsilon (2^{-23}). If we change the exponent to a 1, yielding the value `2.0f`, the value of 1 ULP becomes equal to two times the machine epsilon. And if the exponent is 2, yielding the value `4.0f`, the value of 1 ULP is four

times the machine epsilon. In general, if a floating-point value's unbiased exponent is x , then $1 \text{ ULP} = 2^x \cdot \epsilon$.

The concept of units in the last place illustrates the idea that the precision of a floating-point number depends on its exponent, and is useful for quantifying the error inherent in any floating-point calculation. It can also be useful for finding the floating-point value that is the next largest representable value relative to a known value, or conversely the next smallest representable value relative to that value. This in turn can be useful for converting a greater-than-or-equal comparison into a greater-than comparison. Mathematically, the condition $a \geq b$ is equivalent to the condition $a + 1 \text{ ULP} > b$. We use this little “trick” in the Naughty Dog engine to simplify some logic in our character dialog system. In this system, simple comparisons can be used to select different lines of dialog for the characters to say. Rather than supporting all possible comparison operators, we only support greater-than and less-than checks, and we handle greater-than-or-equal-to and less-than-or-equal-to by adding or subtracting 1 ULP to or from the value being compared.

Impact of Floating-Point Precision on Software

The concepts of limited precision and the machine epsilon have real impacts on game software. For example, let's say we use a floating-point variable to track absolute game time in seconds. How long can we run our game before the magnitude of our clock variable gets so large that adding $1/30^{th}$ of a second to it no longer changes its value? The answer is 12.14 days or 2^{20} seconds. That's longer than most games will be left running, so we can probably get away with using a 32-bit floating-point clock measured in seconds in a game. But clearly it's important to understand the limitations of the floating-point format so that we can predict potential problems and take steps to avoid them when necessary.

IEEE Floating-Point Bit Tricks

See [9, Section 2.1] for a few really useful IEEE floating-point “bit tricks” that can make certain floating-point calculations lightning fast.

3.3.2 Primitive Data Types

C and C++ provide a number of primitive data types. The C and C++ standards provide guidelines on the relative sizes and signedness of these data types, but each compiler is free to define the types slightly differently in order to provide maximum performance on the target hardware.

- `char`. A `char` is usually 8 bits and is generally large enough to hold an ASCII or UTF-8 character (see Section 6.4.4.1). Some compilers define `char` to be signed, while others use unsigned `chars` by default.
- `int`, `short`, `long`. An `int` is supposed to hold a signed integer value that is the most efficient size for the target platform; it is usually defined to be 32 bits wide on a 32-bit CPU architecture, such as Pentium 4 or Xeon, and 64 bits wide on a 64-bit architecture, such as Intel Core i7, although the size of an `int` is also dependent upon other factors such as compiler options and the target operating system. A `short` is intended to be smaller than an `int` and is 16 bits on many machines. A `long` is as large as or larger than an `int` and may be 32 or 64 bits wide, or even wider, again depending on CPU architecture, compiler options and the target OS.
- `float`. On most modern compilers, a `float` is a 32-bit IEEE-754 floating-point value.
- `double`. A `double` is a double-precision (i.e., 64-bit) IEEE-754 floating-point value.
- `bool`. A `bool` is a true/false value. The size of a `bool` varies widely across different compilers and hardware architectures. It is never implemented as a single bit, but some compilers define it to be 8 bits while others use a full 32 bits.

Portable Sized Types

The built-in primitive data types in C and C++ were designed to be portable and therefore nonspecific. However, in many software engineering endeavors, including game engine programming, it is often important to know exactly how wide a particular variable is.

Before C++11, programmers had to rely on non-portable sized types provided by their compiler. For example, the Visual Studio C/C++ compiler defined the following extended keywords for declaring variables that are an explicit number of bits wide: `__int8`, `__int16`, `__int32` and `__int64`. Most other compilers have their own “sized” data types, with similar semantics but slightly different syntax.

Because of these differences between compilers, most game engines achieved source code portability by defining their own custom sized types. For example, at Naughty Dog we use the following sized types:

- `F32` is a 32-bit IEEE-754 floating-point value.
- `U8`, `I8`, `U16`, `I16`, `U32`, `I32`, `U64` and `I64` are unsigned and signed 8-, 16-, 32- and 64-bit integers, respectively.

- U32F and I32F are “fast” unsigned and signed 32-bit values, respectively. Each of these data types contains a value that is at least 32 bits wide, but may be wider if that would result in faster code on the target CPU.

`<cstdint>`

The C++11 standard library introduces a set of standardized sized integer types. They are declared in the `<cstdint>` header, and they include the signed types `std::int8_t`, `std::int16_t`, `std::int32_t` and `std::int64_t` and the unsigned types `std::uint8_t`, `std::uint16_t`, `std::uint32_t` and `std::uint64_t`, along with “fast” variants (like the I32F and U32F types we defined at Naughty Dog). These types free the programmer from having to “wrap” compiler-specific types in order to achieve portability. For a complete list of these sized types, see <http://en.cppreference.com/w/cpp/types/integer>.

OGRE's Primitive Data Types

OGRE defines a number of sized types of its own. `Ogre::uint8`, `Ogre::uint16` and `Ogre::uint32` are the basic unsigned sized integral types.

`Ogre::Real` defines a real floating-point value. It is usually defined to be 32 bits wide (equivalent to a `float`), but it can be redefined globally to be 64 bits wide (like a `double`) by defining the preprocessor macro `OGRE_DOUBLE_PRECISION` to 1. This ability to change the meaning of `Ogre::Real` is generally only used if one's game has a particular requirement for double-precision math, which is rare. Graphics chips (GPUs) always perform their math with 32-bit or 16-bit floats, the CPU/FPU is also usually faster when working in single-precision, and SIMD vector instructions operate on 128-bit registers that contain four 32-bit floats each. Hence, most games tend to stick to single-precision floating-point math.

The data types `Ogre::uchar`, `Ogre::ushort`, `Ogre::uint` and `Ogre::ulong` are just shorthand notations for C/C++'s unsigned `char`, unsigned short and unsigned long, respectively. As such, they are no more or less useful than their native C/C++ counterparts.

The types `Ogre::Radian` and `Ogre::Degree` are particularly interesting. These classes are wrappers around a simple `Ogre::Real` value. The primary role of these types is to permit the angular units of hard-coded literal constants to be documented and to provide automatic conversion between the two unit systems. In addition, the type `Ogre::Angle` represents an angle in the current “default” angle unit. The programmer can define whether the default will be radians or degrees when the OGRE application first starts up.

Perhaps surprisingly, OGRE does not provide a number of sized primitive data types that are commonplace in other game engines. For example, it defines no signed 8-, 16- or 64-bit integral types. If you are writing a game engine on top of OGRE, you will probably find yourself defining these types manually at some point.

3.3.2.1 Multibyte Values and Endianness

Values that are larger than eight bits (one byte) wide are called *multibyte quantities*. They’re commonplace on any software project that makes use of integers and floating-point values that are 16 bits or wider. For example, the integer value 4660 = 0x1234 is represented by the two bytes 0x12 and 0x34. We call 0x12 the most significant byte and 0x34 the least significant byte. In a 32-bit value, such as 0xABCD1234, the most-significant byte is 0xAB and the least-significant is 0x34. The same concepts apply to 64-bit integers and to 32- and 64-bit floating-point values as well.

Multibyte integers can be stored into memory in one of two ways, and different microprocessors may differ in their choice of storage method (see Figure 3.6).

- *Little-endian*. If a microprocessor stores the least significant byte of a multibyte value at a lower memory address than the most significant byte, we say that the processor is *little-endian*. On a little-endian machine, the number 0xABCD1234 would be stored in memory using the consecutive bytes 0x34, 0x12, 0xCD, 0xAB.
- *Big-endian*. If a microprocessor stores the most significant byte of a multibyte value at a lower memory address than the least significant byte,

```
U32 value = 0xABCD1234;  
U8* pBytes = (U8*)&value;
```

Big-endian		Little-endian	
pBytes + 0x0	0xAB	pBytes + 0x0	0x34
pBytes + 0x1	0xCD	pBytes + 0x1	0x12
pBytes + 0x2	0x12	pBytes + 0x2	0xCD
pBytes + 0x3	0x34	pBytes + 0x3	0xAB

Figure 3.6. Big- and little-endian representations of the value 0xABCD1234.

we say that the processor is *big-endian*. On a big-endian machine, the number 0xABCD1234 would be stored in memory using the bytes 0xAB, 0xCD, 0x12, 0x34.

Most programmers don't need to think much about endianness. However, when you're a game programmer, endianness can become a bit of a thorn in your side. This is because games are usually *developed* on a Windows or Linux machine running an Intel Pentium processor (which is little-endian), but *run* on a console such as the Wii, Xbox 360 or PlayStation 3—all three of which utilize a variant of the PowerPC processor (which can be configured to use either endianness, but is big-endian by default). Now imagine what happens when you generate a data file for consumption by your game engine on an Intel processor and then try to load that data file into your engine running on a PowerPC processor. Any multibyte value that you wrote out into that data file will be stored in little-endian format. But when the game engine reads the file, it expects all of its data to be in big-endian format. The result? You'll write 0xABCD1234, but you'll read 0x3412CDAB, and that's clearly not what you intended!

There are at least two solutions to this problem.

1. You could write all your data files as text and store all multibyte numbers as sequences of decimal or hexadecimal digits, one character (one byte) per digit. This would be an inefficient use of disk space, but it would work.
2. You can have your tools endian-swap the data prior to writing it into a binary data file. In effect, you make sure that the data file uses the endianness of the target microprocessor (the game console), even if the tools are running on a machine that uses the opposite endianness.

Integer Endian-Swapping

Endian-swapping an integer is not conceptually difficult. You simply start at the most significant byte of the value and swap it with the least significant byte; you continue this process until you reach the halfway point in the value. For example, 0xA7891023 would become 0x231089A7.

The only tricky part is knowing *which bytes* to swap. Let's say you're writing the contents of a C struct or C++ class from memory out to a file. To properly endian-swap this data, you need to keep track of the locations and sizes of each data member in the struct and swap each one appropriately based on its size. For example, the structure


```

struct Example
{
    U32    m_a;
    U16    m_b;
    U32    m_c;
};

```

might be written out to a data file as follows:

```

void writeExampleStruct(Example& ex, Stream& stream)
{
    stream.writeU32(swapU32(ex.m_a));
    stream.writeU16(swapU16(ex.m_b));
    stream.writeU32(swapU32(ex.m_c));
}

```

and the swap functions might be defined like this:

```

inline U16 swapU16(U16 value)
{
    return ((value & 0x00FF) << 8)
        | ((value & 0xFF00) >> 8);
}

inline U32 swapU32(U32 value)
{
    return ((value & 0x000000FF) << 24)
        | ((value & 0x0000FF00) << 8)
        | ((value & 0x00FF0000) >> 8)
        | ((value & 0xFF000000) >> 24);
}

```

You cannot simply cast the `Example` object into an array of bytes and blindly swap the bytes using a single general-purpose function. We need to know both *which data members* to swap and *how wide* each member is, and each data member must be swapped individually.

Some compilers provide built-in endian-swapping macros, freeing you from having to write your own. For example, gcc offers a family of macros named `__builtin_bswapXX()` for performing 16-, 32- and 64-bit endian swaps. However, such compiler-specific facilities are of course non-portable.

Floating-Point Endian-Swapping

As we've seen, an IEEE-754 floating-point value has a detailed internal structure involving some bits for the mantissa, some bits for the exponent and a sign bit. However, you can endian-swap it just as if it were an integer, because bytes are bytes. You merely reinterpret the bit pattern of your `float`

as if it were a `std::int32_t`, perform the endian swapping operation, and then reinterpret the result as a `float` again.

You can reinterpret floats as integers by using C++’s `reinterpret_cast` operator on a pointer to the float, and then dereferencing the type-cast pointer; this is known as *type punning*. But punning can lead to optimization bugs when strict aliasing is enabled. (See <http://www.cocoawithlove.com/2008/04/using-pointers-to-recast-in-c-is-bad.html> for an excellent description of this problem.) One alternative that’s guaranteed to be portable is to use a union, as follows:

```
union U32F32
{
    U32    m_asU32;
    F32    m_asF32;
};

inline F32 swapF32(F32 value)
{
    U32F32 u;
    u.m_asF32 = value;

    // endian-swap as integer
    u.m_asU32 = swapU32(u.m_asU32);

    return u.m_asF32;
}
```

3.3.3 Kilobytes versus Kibibytes

You’ve probably used metric (SI) units like kilobytes (kB) and megabytes (MB) to describe quantities of memory. However, the use of these units to describe quantities of memory that are measured in powers of two isn’t strictly correct. When a computer programmer speaks of a “kilobyte,” she or he usually means 1024 bytes. But SI units define the prefix “kilo” to mean 10^3 or 1000, not 1024.

To resolve this ambiguity, the International Electrotechnical Commission (IEC) in 1998 established a new set of SI-like prefixes for use in computer science. These prefixes are defined in terms of powers of two rather than powers of ten, so that computer engineers can precisely and conveniently specify quantities that are powers of two. In this new system, instead of kilobyte (1000 bytes), we say kibibyte (1024 bytes, abbreviated KiB). And instead of megabyte (1,000,000 bytes), we say mebibyte ($1024 \times 1024 = 1,048,576$ bytes, abbreviated MiB). Table 3.1 summarizes the sizes, prefixes and names of the

Metric (SI)			IEC		
Value	Unit	Name	Value	Unit	Name
1000	kB	kilobyte	1024	KiB	kibibyte
1000 ²	MB	megabyte	1024 ²	MiB	mebibyte
1000 ³	GB	gigabyte	1024 ³	GiB	gibibyte
1000 ⁴	TB	terabyte	1024 ⁴	TiB	tebibyte
1000 ⁵	PB	petabyte	1024 ⁵	PiB	pebibyte
1000 ⁶	EB	exabyte	1024 ⁶	EiB	exbibyte
1000 ⁷	ZB	zettabyte	1024 ⁷	ZiB	zebibyte
1000 ⁸	YB	yottabyte	1024 ⁸	YiB	yobibyte

Table 3.1. Comparison of metric (SI) units and IEC units for describing quantities of bytes.

most commonly used byte quantity units in both the SI and IEC systems. We'll use IEC units throughout this book.

3.3.4 Declarations, Definitions and Linkage

3.3.4.1 Translation Units Revisited

As we saw in Chapter 2, a C or C++ program is comprised of *translation units*. The compiler translates one .cpp file at a time, and for each one it generates an output file called an object file (.o or .obj). A .cpp file is the smallest unit of translation operated on by the compiler; hence, the name “translation unit.” An object file contains not only the compiled machine code for all of the functions defined in the .cpp file, but also all of its global and static variables. In addition, an object file may contain *unresolved references* to functions and global variables defined in *other* .cpp files.

The compiler only operates on one translation unit at a time, so whenever it encounters a reference to an external global variable or function, it must “go on faith” and assume that the entity in question really exists, as shown in Figure 3.7. It is the linker’s job to combine all of the object files into a final executable image. In doing so, the linker reads all of the object files and attempts to resolve all of the unresolved cross-references between them. If it is successful, an executable image is generated containing all of the functions, global variables and static variables, with all cross-translation-unit references properly resolved. This is depicted in Figure 3.8.

The linker’s primary job is to resolve external references, and in this

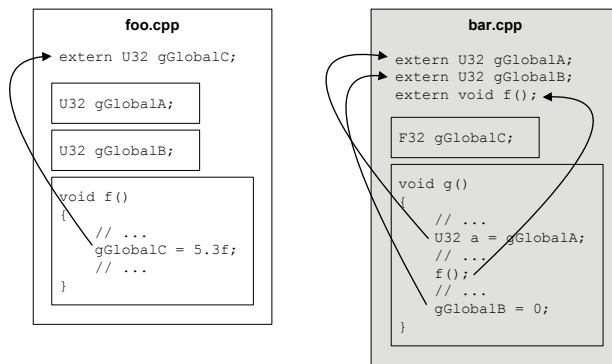


Figure 3.7. Unresolved external references in two translation units.

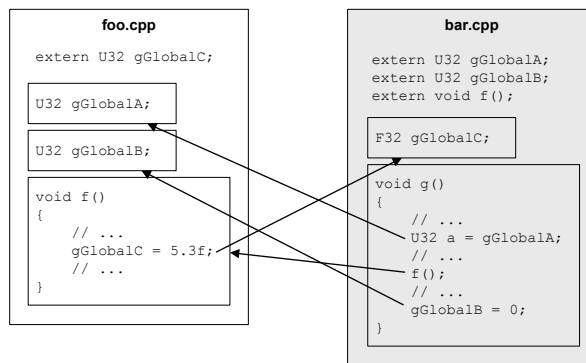


Figure 3.8. Fully resolved external references after successful linking.

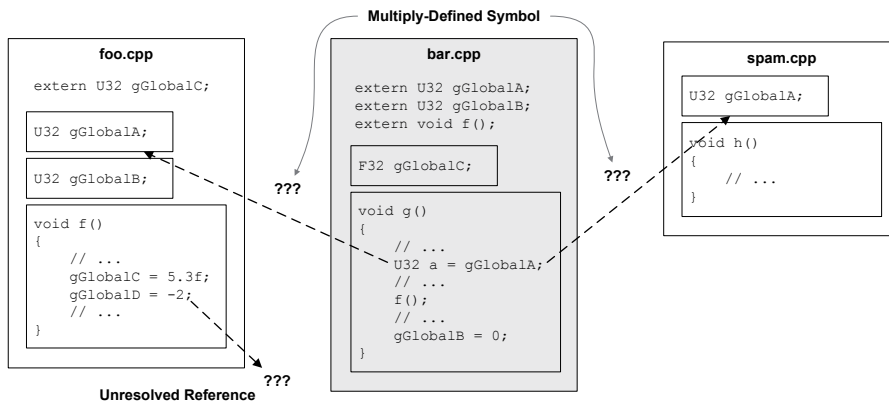


Figure 3.9. The two most common linker errors.

capacity it can generate only two kinds of errors:

1. The target of an `extern` reference might not be found, in which case the linker generates an “unresolved symbol” error.
2. The linker might find more than one variable or function with the same name, in which case it generates a “multiply defined symbol” error.

These two situations are shown in Figure 3.9.

3.3.4.2 Declaration versus Definition

In the C and C++ languages, variables and functions must be *declared* and *defined* before they can be used. It is important to understand the difference between a declaration and a definition in C and C++.

- A *declaration* is a description of a data object or function. It provides the compiler with the *name* of the entity and its *data type* or *function signature* (i.e., return type and argument type(s)).
- A *definition*, on the other hand, describes a unique region of memory in the program. This memory might contain a variable, an instance of a struct or class or the machine code of a function.

In other words, a declaration is a *reference to an entity*, while a definition is the *entity itself*. A definition is always a declaration, but the reverse is not always the case—it is possible to write a pure declaration in C and C++ that is not a definition.

Functions are *defined* by writing the body of the function immediately after the signature, enclosed in curly braces:

foo.cpp

```
// definition of the max() function
int max(int a, int b)
{
    return (a > b) ? a : b;
}

// definition of the min() function
int min(int a, int b)
{
    return (a <= b) ? a : b;
}
```

A pure declaration can be provided for a function so that it can be used in other translation units (or later in the same translation unit). This is done by writing a function signature followed by a semicolon, with an optional prefix of `extern`:

foo.h

```
extern int max(int a, int b); // a function declaration

int min(int a, int b); // also a declaration (the extern
                        // is optional/assumed)
```

Variables and instances of classes and `structs` are defined by writing the data type followed by the name of the variable or instance and an optional array specifier in square brackets:

foo.cpp

```
// All of these are variable definitions:
U32 gGlobalInteger = 5;
F32 gGlobalFloatArray[16];
MyClass gGlobalInstance;
```

A global variable defined in one translation unit can optionally be *declared* for use in other translation units by using the `extern` keyword:

foo.h

```
// These are all pure declarations:
extern U32 gGlobalInteger;
extern F32 gGlobalFloatArray[16];
extern MyClass gGlobalInstance;
```

Multiplicity of Declarations and Definitions

Not surprisingly, any particular data object or function in a C/C++ program can have multiple identical *declarations*, but each can have only one *definition*. If two or more identical definitions exist in a single translation unit, the compiler will notice that multiple entities have the same name and flag an error. If two or more identical definitions exist in *different* translation units, the compiler will not be able to identify the problem, because it operates on one translation unit at a time. But in this case, the linker will give us a “multiply defined symbol” error when it tries to resolve the cross-references.

Definitions in Header Files and Inlining

It is usually dangerous to place *definitions* in header files. The reason for this should be pretty obvious: if a header file containing a *definition* is `#included` into more than one `.cpp` file, it's a sure-fire way of generating a "multiply defined symbol" linker error.

Inline function definitions are an exception to this rule, because each invocation of an inline function gives rise to a brand new copy of that function's machine code, embedded directly into the calling function. In fact, inline function definitions *must* be placed in header files if they are to be used in more than one translation unit. Note that it is *not* sufficient to tag a function *declaration* with the `inline` keyword in a `.h` file and then place the body of that function in a `.cpp` file. The compiler must be able to "see" the body of the function in order to inline it. For example:

foo.h

```
// This function definition will be inlined properly.
inline int max(int a, int b)
{
    return (a > b) ? a : b;
}

// This declaration cannot be inlined because the
// compiler cannot "see" the body of the function.
inline int min(int a, int b);
```

foo.cpp

```
// The body of min() is effectively "hidden" from the
// compiler, so it can ONLY be inlined within foo.cpp.
int min(int a, int b)
{
    return (a <= b) ? a : b;
}
```

The `inline` keyword is really just a hint to the compiler. It does a cost/benefit analysis of each inline function, weighing the size of the function's code versus the potential performance benefits of inlining it, and the compiler gets the final say as to whether the function will really be inlined or not. Some compilers provide syntax like `__forceinline`, allowing the programmer to bypass the compiler's cost/benefit analysis and control function inlining directly.

Templates and Header Files

The definition of a templated class or function must be visible to the compiler across all translation units in which it is used. As such, if you want a template to be usable in more than one translation unit, the template must be placed into a header file (just as inline function definitions must be). The declaration and definition of a template are therefore inseparable: You cannot declare templated functions or classes in a header but “hide” their definitions inside a .cpp file, because doing so would render those definitions invisible within any other .cpp file that includes that header.

3.3.4.3 Linkage

Every definition in C and C++ has a property known as *linkage*. A definition with *external linkage* is visible to and can be referenced by translation units other than the one in which it appears. A definition with *internal linkage* can only be “seen” inside the translation unit in which it appears and thus cannot be referenced by other translation units. We call this property *linkage* because it dictates whether or not the linker is permitted to cross-reference the entity in question. So, in a sense, linkage is the translation unit’s equivalent of the `public:` and `private:` keywords in C++ class definitions.

By default, definitions have external linkage. The `static` keyword is used to change a definition’s linkage to internal. Note that two or more identical static definitions in two or more different .cpp files are considered to be *distinct entities* by the linker (just as if they had been given different names), so they will *not* generate a “multiply defined symbol” error. Here are some examples:

foo.cpp

```
// This variable can be used by other .cpp files
// (external linkage).
U32 gExternalVariable;

// This variable is only usable within foo.cpp (internal
// linkage).
static U32 gInternalVariable;

// This function can be called from other .cpp files
// (external linkage).
void externalFunction()
{
    // ...
}
```



```
// This function can only be called from within foo.cpp
// (internal linkage).
static void internalFunction()
{
    // ...
}
```

bar.cpp

```
// This declaration grants access to foo.cpp's variable.
extern U32 gExternalVariable;

// This 'gInternalVariable' is distinct from the one
// defined in foo.cpp -- no error. We could just as
// well have named it gInternalVariableForBarCpp -- the
// net effect is the same.
static U32 gInternalVariable;

// This function is distinct from foo.cpp's
// version -- no error. It acts as if we had named it
// internalFunctionForBarCpp().
static void internalFunction()
{
    // ...
}

// ERROR -- multiply defined symbol!
void externalFunction()
{
    // ...
}
```

Technically speaking, *declarations* don't have a linkage property at all, because they do not allocate any storage in the executable image; therefore, there is no question as to whether or not the linker should be permitted to cross-reference that storage. A declaration is merely a reference to an entity defined elsewhere. However, it is sometimes convenient to speak about declarations as having internal linkage, because a declaration only applies to the translation unit in which it appears. If we allow ourselves to loosen our terminology in this manner, then declarations *always* have internal linkage—there is no way to cross-reference a single declaration in multiple .cpp files. (If we put a declaration in a header file, then multiple .cpp files can “see” that declaration, but they are in effect each getting a distinct *copy* of the declaration, and each copy has internal linkage within that translation unit.)

This leads us to the real reason why inline function definitions are permitted in header files: it is because inline functions have *internal linkage* by default, just as if they had been declared `static`. If multiple `.cpp` files `#include` a header containing an inline function definition, each translation unit gets a private copy of that function's body, and no "multiply defined symbol" errors are generated. The linker sees each copy as a distinct entity.

3.3.5 Memory Layout of a C/C++ Program

A program written in C or C++ stores its data in a number of different places in memory. In order to understand how storage is allocated and how the various types of C/C++ variables work, we need to understand the memory layout of a C/C++ program.

3.3.5.1 Executable Image

When a C/C++ program is built, the linker creates an *executable file*. Most UNIX-like operating systems, including many game consoles, employ a popular executable file format called the *executable and linking format* (ELF). Executable files on those systems therefore have a `.elf` extension. The Windows executable format is similar to the ELF format; executables under Windows have a `.exe` extension. Whatever its format, the executable file always contains a partial *image* of the program as it will exist in memory when it runs. I say a "partial" image because the program generally allocates memory at runtime in addition to the memory laid out in its executable image.

The executable image is divided into contiguous blocks called *segments* or *sections*. Every operating system lays things out a little differently, and the layout may also differ slightly from executable to executable on the same operating system. But the image is usually comprised of at least the following four segments:

1. *Text segment*. Sometimes called the *code segment*, this block contains executable machine code for all functions defined by the program.
2. *Data segment*. This segment contains all *initialized* global and static variables. The memory needed for each global variable is laid out exactly as it will appear when the program is run, and the proper initial values are all filled in. So when the executable file is loaded into memory, the initialized global and static variables are ready to go.
3. *BSS segment*. "BSS" is an outdated name which stands for "block started by symbol." This segment contains all of the *uninitialized* global and

static variables defined by the program. The C and C++ languages explicitly define the initial value of any uninitialized global or static variable to be zero. But rather than storing a potentially very large block of zeros in the BSS section, the linker simply stores a *count* of how many zero bytes are required to account for all of the uninitialized globals and statics in the segment. When the executable is loaded into memory, the operating system reserves the requested number of bytes for the BSS section and fills it with zeros prior to calling the program's entry point (e.g., `main()` or `WinMain()`).

4. *Read-only data segment.* Sometimes called the *rodata* segment, this segment contains any read-only (constant) global data defined by the program. For example, all floating-point constants (e.g., `const float kPi = 3.141592f;`) and all global object instances that have been declared with the `const` keyword (e.g., `const Foo gReadOnlyFoo;`) reside in this segment. Note that integer constants (e.g., `const int kMaxMonsters = 255;`) are often used as *manifest constants* by the compiler, meaning that they are inserted directly into the machine code wherever they are used. Such constants occupy storage in the text segment, but they are not present in the read-only data segment.

Global variables (variables defined at *file scope* outside any function or class declaration) are stored in either the data or BSS segments, depending on whether or not they have been initialized. The following global will be stored in the data segment, because it has been initialized:

foo.cpp

```
F32 gInitializedGlobal = -2.0f;
```

and the following global will be allocated and initialized to zero by the operating system, based on the specifications given in the BSS segment, because it has not been initialized by the programmer:

foo.cpp

```
F32 gUninitializedGlobal;
```

We've seen that the `static` keyword can be used to give a global variable or function definition *internal linkage*, meaning that it will be "hidden" from other translation units. The `static` keyword can also be used to declare a global variable *within a function*. A function-static variable is *lexically scoped* to the function in which it is declared (i.e., the variable's name can only be "seen" inside the function). It is initialized the first time the function is called (rather than before `main()` is called, as with file-scope statics). But in terms of memory layout in the executable image, a function-static variable acts identically

to a file-static global variable—it is stored in either the data or BSS segment based on whether or not it has been initialized.

```
void readHitchhikersGuide(U32 book)
{
    static U32 sBooksInTheTrilogy = 5; // data segment
    static U32 sBooksRead;             // BSS segment
    // ...
}
```

3.3.5.2 Program Stack

When an executable program is loaded into memory and run, the operating system reserves an area of memory for the *program stack*. Whenever a function is called, a contiguous area of stack memory is pushed onto the stack—we call this block of memory a *stack frame*. If function `a()` calls another function `b()`, a new stack frame for `b()` is pushed on top of `a()`'s frame. When `b()` returns, its stack frame is popped, and execution continues wherever `a()` left off.

A stack frame stores three kinds of data:

1. It stores the *return address* of the calling function so that execution may continue in the calling function when the called function returns.
2. The contents of all relevant *CPU registers* are saved in the stack frame. This allows the new function to use the registers in any way it sees fit, without fear of overwriting data needed by the calling function. Upon return to the calling function, the state of the registers is restored so that execution of the calling function may resume. The return value of the called function, if any, is usually left in a specific register so that the calling function can retrieve it, but the other registers are restored to their original values.
3. The stack frame also contains all *local variables* declared by the function; these are also known as *automatic variables*. This allows each distinct function invocation to maintain its own private copy of every local variable, even when a function calls itself recursively. (In practice, some local variables are actually allocated to CPU registers rather than being stored in the stack frame, but for the most part such variables operate as if they were allocated within the function's stack frame.)

Pushing and popping stack frames is usually implemented by adjusting the value of a single register in the CPU, known as the stack pointer. Figure 3.10 illustrates what happens when the functions shown below are executed.

```
void c()  
{  
    U32 localC1;  
    // ...  
}  
  
F32 b()  
{  
    F32 localB1;  
    I32 localB2;  
  
    // ...  
  
    c();  
  
    // ...  
  
    return localB1;  
}
```

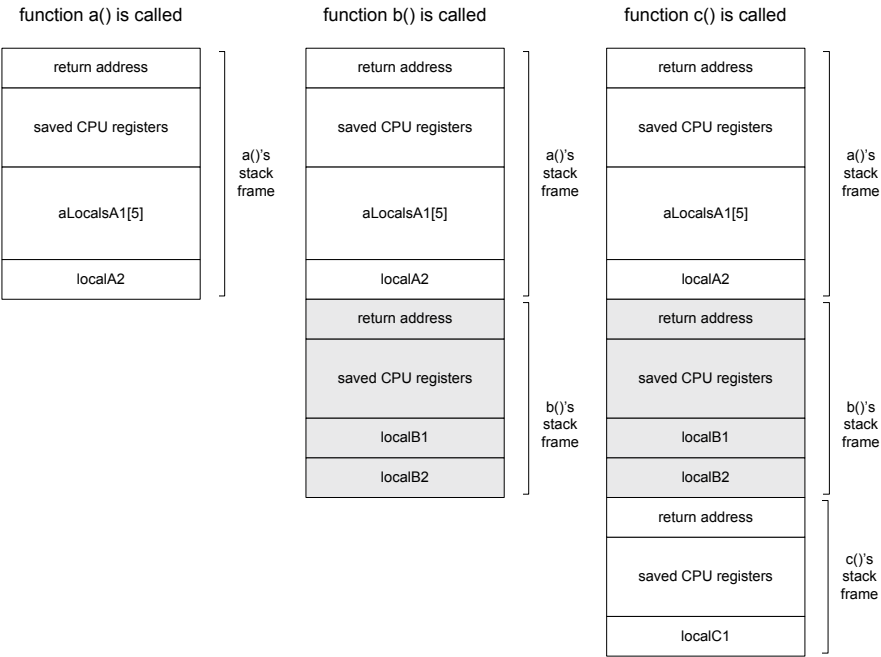


Figure 3.10. Stack frames.

```
void a()
{
    U32 aLocalsA1[5];

    // ...

    F32 localA2 = b();

    // ...
}
```

When a function containing automatic variables returns, its stack frame is abandoned and all automatic variables in the function should be treated as if they no longer exist. Technically, the memory occupied by those variables is still there in the abandoned stack frame—but that memory will very likely be overwritten as soon as another function is called. A common error involves returning the address of a local variable, like this:

```
U32* getMeaningOfLife()
{
    U32 anInteger = 42;
    return &anInteger;
}
```

You *might* get away with this if you use the returned pointer immediately and don't call any other functions in the interim. But more often than not, this kind of code will crash—sometimes in ways that can be difficult to debug.

3.3.5.3 Dynamic Allocation Heap

Thus far, we've seen that a program's data can be stored as global or static variables or as local variables. The globals and statics are allocated within the executable image, as defined by the data and BSS segments of the executable file. The locals are allocated on the program stack. Both of these kinds of storage are *statically* defined, meaning that the size and layout of the memory is known when the program is compiled and linked. However, a program's memory requirements are often not fully known at compile time. A program usually needs to allocate additional memory *dynamically*.

To allow for dynamic allocation, the operating system maintains a block of memory for each running process from which memory can be allocated by calling `malloc()` (or an OS-specific function like `HeapAlloc()` under Windows) and later returned for reuse by the process at some future time by calling `free()` (or an OS-specific function like `HeapFree()`). This memory block is

known as *heap memory*, or the *free store*. When we allocate memory dynamically, we sometimes say that this memory resides *on the heap*.

In C++, the global `new` and `delete` operators are used to allocate and free memory to and from the free store. Be wary, however—individual classes may overload these operators to allocate memory in custom ways, and even the *global* `new` and `delete` operators can be overloaded, so you cannot simply assume that `new` is always allocating from the global heap.

We will discuss dynamic memory allocation in more depth in Chapter 7. For additional information, see http://en.wikipedia.org/wiki/Dynamic_memory_allocation.

3.3.6 Member Variables

C structs and C++ classes allow variables to be grouped into logical units. It's important to remember that a `class` or `struct` *declaration* allocates no memory. It is merely a description of the layout of the data—a cookie cutter which can be used to stamp out *instances* of that `struct` or `class` later on. For example:

```
struct Foo // struct declaration
{
    U32  mUnsignedValue;
    F32  mFloatValue;
    bool mBooleanValue;
};
```

Once a `struct` or `class` has been declared, it can be allocated (defined) in any of the ways that a primitive data type can be allocated; for example,

- as an automatic variable, on the program stack;

```
void someFunction()
{
    Foo localFoo;
    // ...
}
```

- as a global, file-static or function-static;

```
Foo gFoo;
static Foo sFoo;

void someFunction()
{
    static Foo sLocalFoo;
    // ...
}
```

- dynamically allocated from the heap. In this case, the pointer or reference variable used to hold the address of the data can itself be allocated as an automatic, global, static or even dynamically.

```

Foo* gpFoo = nullptr; // global pointer to a Foo

void someFunction()
{
    // allocate a Foo instance from the heap
    gpFoo = new Foo;

    // ...

    // allocate another Foo, assign to local pointer
    Foo* pAnotherFoo = new Foo;

    // ...

    // allocate a POINTER to a Foo from the heap
    Foo** ppFoo = new Foo*;
    (*ppFoo) = pAnotherFoo;
}

```

3.3.6.1 Class-Static Members

As we've seen, the `static` keyword has many different meanings depending on context:

- When used at file scope, `static` means “restrict the visibility of this variable or function so it can only be seen inside this .cpp file.”
- When used at function scope, `static` means “this variable is a global, not an automatic, but it can only be seen inside this function.”
- When used inside a `struct` or `class` declaration, `static` means “this variable is not a regular member variable, but instead acts just like a global.”

Notice that when `static` is used inside a class declaration, it does not control the *visibility* of the variable (as it does when used at file scope)—rather, it differentiates between regular per-instance member variables and per-class variables that act like globals. The *visibility* of a class-static variable is determined by the use of `public:`, `protected:` or `private:` keywords in the class declaration. Class-static variables are automatically included within the namespace of the class or struct in which they are declared. So the name of the class or struct must be used to disambiguate the variable whenever it is used outside that class or struct (e.g., `Foo::sVarName`).

Like an extern declaration for a regular global variable, the declaration of a class-static variable within a class allocates no memory. The memory for the class-static variable must be defined in a .cpp file. For example:

foo.h

```
class Foo
{
public:
    static F32 sClassStatic; // allocates no
                           // memory!
};
```

foo.cpp

```
F32 Foo::sClassStatic = -1.0f; // define memory and
                               // initialize
```

3.3.7 Object Layout in Memory

It’s useful to be able to visualize the memory layout of your classes and structs. This is usually pretty straightforward—we can simply draw a box for the struct or class, with horizontal lines separating data members. An example of such a diagram for the struct `Foo` listed below is shown in Figure 3.11.

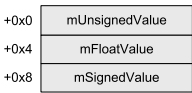


Figure 3.11. Memory layout of a simple struct.

```
struct Foo
{
    U32    mUnsignedValue;
    F32    mFloatValue;
    I32    mSignedValue;
};
```

The sizes of the data members are important and should be represented in your diagrams. This is easily done by using the width of each data member to indicate its size in bits—i.e., a 32-bit integer should be roughly four times the width of an eight-bit integer (see Figure 3.12).

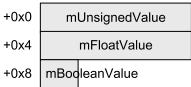


Figure 3.12. A memory layout using width to indicate member sizes.

```
struct Bar
{
    U32    mUnsignedValue;
    F32    mFloatValue;
    bool   mBooleanValue; // diagram assumes this is 8 bits
};
```

3.3.7.1 Alignment and Packing

As we start to think more carefully about the layout of our structs and classes in memory, we may start to wonder what happens when small data members are interspersed with larger members. For example:

```
struct InefficientPacking
{
    U32    mU1; // 32 bits
    F32    mF2; // 32 bits
    U8     mB3; // 8 bits
    I32    mI4; // 32 bits
    bool   mB5; // 8 bits
    char*  mP6; // 32 bits
};
```

You might imagine that the compiler simply packs the data members into memory as tightly as it can. However, this is not usually the case. Instead, the compiler will typically leave “holes” in the layout, as depicted in Figure 3.13. (Some compilers can be requested not to leave these holes by using a preprocessor directive like `#pragma pack`, or via command-line options; but the default behavior is to space out the members as shown in Figure 3.13.)

Why does the compiler leave these “holes”? The reason lies in the fact that every data type has a natural *alignment*, which must be respected in order to permit the CPU to read and write memory effectively. The *alignment* of a data object refers to whether its *address in memory* is a multiple of its *size* (which is generally a power of two):

- An object with 1-byte alignment resides at any memory address.
- An object with 2-byte alignment resides only at even addresses (i.e., addresses whose least significant nibble is 0x0, 0x2, 0x4, 0x8, 0xA, 0xC or 0xE).
- An object with 4-byte alignment resides only at addresses that are a multiple of four (i.e., addresses whose least significant nibble is 0x0, 0x4, 0x8 or 0xC).
- A 16-byte aligned object resides only at addresses that are a multiple of 16 (i.e., addresses whose least significant nibble is 0x0).

Alignment is important because many modern processors can actually only read and write properly aligned blocks of data. For example, if a program requests that a 32-bit (4-byte) integer be read from address 0x6A341174, the memory controller will load the data happily because the address is 4-byte aligned (in this case, its least significant nibble is 0x4). However, if a request is made to load a 32-bit integer from address 0x6A341173, the memory controller now has to read *two* 4-byte blocks: the one at 0x6A341170 and the one

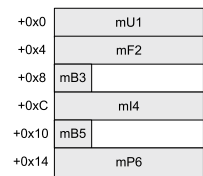


Figure 3.13. Inefficient struct packing due to mixed data member sizes.

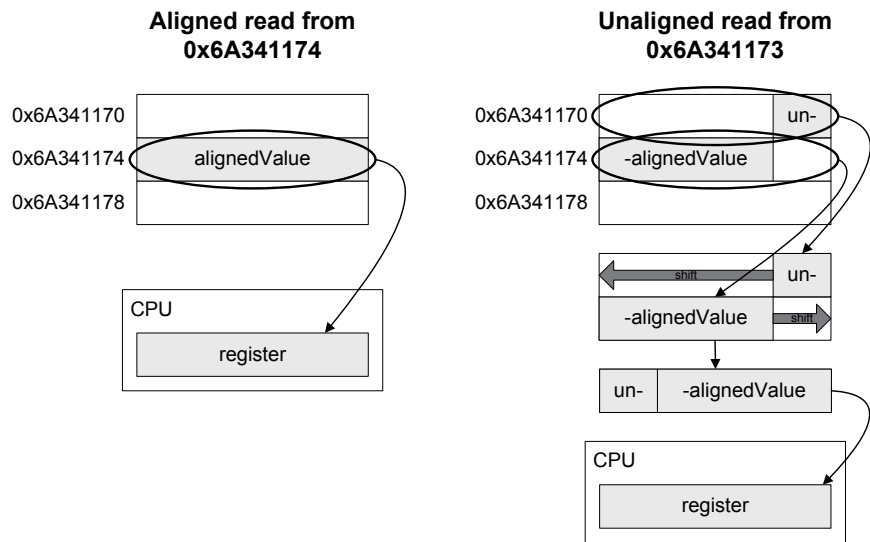


Figure 3.14. Aligned and unaligned reads of a 32-bit integer.

at 0x6A341174. It must then mask and shift the two parts of the 32-bit integer and logically OR them together into the destination register on the CPU. This is shown in Figure 3.14.

Some microprocessors don’t even go this far. If you request a read or write of unaligned data, you might just get garbage. Or your program might just crash altogether! (The PlayStation 2 is a notable example of this kind of intolerance for unaligned data.)

Different data types have different alignment requirements. A good rule of thumb is that a data type should be aligned to a boundary equal to the width of the data type in bytes. For example, 32-bit values generally have a 4-byte alignment requirement, 16-bit values should be 2-byte aligned, and 8-bit values can be stored at any address (1-byte aligned). On CPUs that support SIMD vector math, the vector registers each contain four 32-bit floats, for a total of 128 bits or 16 bytes. And as you would guess, a four-float SIMD vector typically has a 16-byte alignment requirement.

This brings us back to those “holes” in the layout of `struct InefficientPacking` shown in Figure 3.13. When smaller data types like 8-bit bools are interspersed with larger types like 32-bit integers or floats in a structure or class, the compiler introduces padding (holes) in order to ensure that everything is properly aligned. It’s a good idea to think about alignment and packing when declaring your data structures. By simply rearranging the members of `struct InefficientPacking` from the example above, we can eliminate some of the wasted padding space, as shown below and in Figure 3.15:

+0x0	mU1		
+0x4	mF2		
+0x8	mI4		
+0xC	mP6		
+0x10	mB3	mB5	(pad)

Figure 3.15. More efficient packing by grouping small members together.

```

struct MoreEfficientPacking
{
    U32    mU1; // 32 bits (4-byte aligned)
    F32    mF2; // 32 bits (4-byte aligned)
    I32    mI4; // 32 bits (4-byte aligned)
    char*  mP6; // 32 bits (4-byte aligned)
    U8     mB3; // 8 bits (1-byte aligned)
    bool   mB5; // 8 bits (1-byte aligned)
};

```

You'll notice in Figure 3.15 that the size of the structure as a whole is now 20 bytes, not 18 bytes as we might expect, because it has been padded by two bytes at the end. This padding is added by the compiler to ensure proper alignment of the structure in an *array context*. That is, if an array of these structs is defined and the first element of the array is aligned, then the padding at the end guarantees that *all subsequent elements* will also be aligned properly.

The alignment of a structure as a whole is equal to the largest alignment requirement among its members. In the example above, the largest member alignment is 4-byte, so the structure as a whole should be 4-byte aligned. I usually like to add explicit padding to the end of my structs to make the wasted space visible and explicit, like this:

```

struct BestPacking
{
    U32    mU1;      // 32 bits (4-byte aligned)
    F32    mF2;      // 32 bits (4-byte aligned)
    I32    mI4;      // 32 bits (4-byte aligned)
    char*  mP6;      // 32 bits (4-byte aligned)
    U8     mB3;      // 8 bits (1-byte aligned)
    bool   mB5;      // 8 bits (1-byte aligned)
    U8     _pad[2]; // explicit padding
};

```

3.3.7.2 Memory Layout of C++ Classes

Two things make C++ classes a little different from C structures in terms of memory layout: *inheritance* and *virtual functions*.

When class B inherits from class A, B's data members simply appear immediately after A's in memory, as shown in Figure 3.16. Each new derived class simply tacks its data members on at the end, although alignment requirements may introduce padding between the classes. (Multiple inheritance does some whacky things, like including multiple copies of a single base class in the memory layout of a derived class. We won't cover the details here, because game programmers usually prefer to avoid multiple inheritance altogether anyway.)

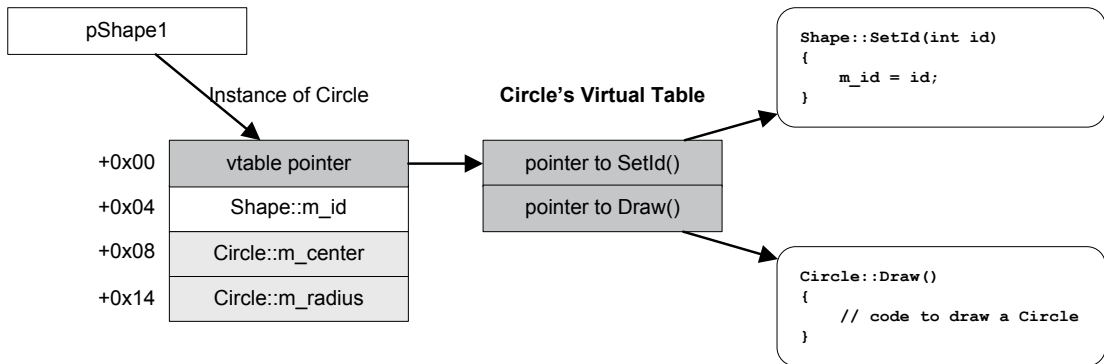


Figure 3.16. Effect of inheritance on class layout.

If a class contains or inherits one or more *virtual functions*, then four additional bytes (or eight bytes if the target hardware uses 64-bit addresses) are added to the class layout, typically at the very beginning of the class' layout. These four or eight bytes are collectively called the *virtual table pointer* or *vpointer*, because they contain a pointer to a data structure known as the *virtual function table* or *vtable*. The vtable for a particular class contains pointers to all the virtual functions that it declares or inherits. Each concrete class has its own virtual table, and every instance of that class has a pointer to it, stored in its vpointer.

The virtual function table is at the heart of polymorphism, because it allows code to be written that is ignorant of the specific concrete classes it is dealing with. Returning to the ubiquitous example of a `Shape` base class with derived classes for `Circle`, `Rectangle` and `Triangle`, let's imagine that `Shape` defines a virtual function called `Draw()`. The derived classes all override this function, providing distinct implementations named `Circle::Draw()`, `Rectangle::Draw()` and `Triangle::Draw()`. The virtual table for any class derived from `Shape` will contain an entry for the `Draw()` function, but that entry will point to different function implementations, depending on the concrete class. `Circle`'s vtable will contain a pointer to `Circle::Draw()`, while `Rectangle`'s virtual table will point to `Rectangle::Draw()`, and `Triangle`'s vtable will point to `Triangle::Draw()`. Given an arbitrary pointer to a `Shape` (`Shape* pShape`), the code can simply dereference the vtable pointer, look up the `Draw()` function's entry in the vtable, and call it. The result will be to call `Circle::Draw()` when `pShape` points to an instance of `Circle`, `Rectangle::Draw()` when `pShape` points to a `Rectangle`, and `Triangle::Draw()` when `pShape` points to a `Triangle`.

These ideas are illustrated by the following code excerpt. Notice that the base class `Shape` defines two virtual functions, `SetId()` and `Draw()`, the latter of which is declared to be *pure virtual*. (This means that `Shape` provides no default implementation of the `Draw()` function, and derived classes *must* override it if they want to be instantiable.) Class `Circle` derives from `Shape`, adds some data members and functions to manage its center and radius, and overrides the `Draw()` function; this is depicted in Figure 3.17. Class `Triangle` also derives from `Shape`. It adds an array of `Vector3` objects to store its three vertices and adds some functions to get and set the individual vertices. Class `Triangle` overrides `Draw()` as we'd expect, and for illustrative purposes it also overrides `SetId()`. The memory image generated by the `Triangle` class is shown in Figure 3.18.

Figure 3.17. `pShape1` points to an instance of class `Circle`.

```
class Shape
{
public:
    virtual void SetId(int id) { m_id = id; }
    int GetId() const { return m_id; }

    virtual void Draw() = 0; // pure virtual -- no impl.

private:
    int m_id;
};

class Circle : public Shape
{
public:
    void SetCenter(const Vector3& c) { m_center=c; }
    Vector3 GetCenter() const { return m_center; }

    void SetRadius(float r) { m_radius = r; }
    float GetRadius() const { return m_radius; }

    virtual void Draw()
    {
        // code to draw a circle
    }

private:
    Vector3 m_center;
    float m_radius;
};
```